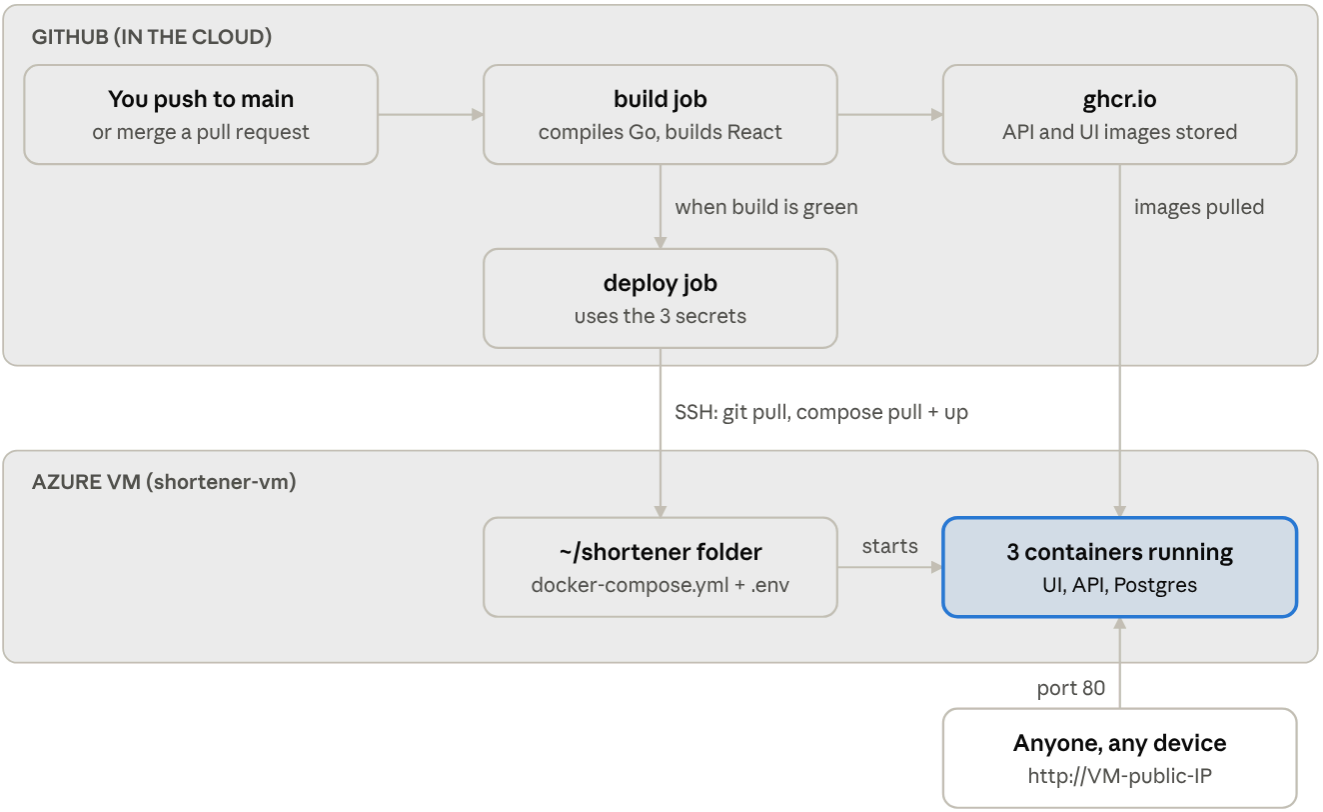
Deploy the URL Shortener on Azure

Nov 6, 2026 · @Amit

How it works

An Azure VM runs your three containers all day, with a public IP anyone can open. Every push to `main` builds new images on GitHub, then a second job logs in to the VM over SSH and swaps in the new containers.

GitHub builds the images, then deploys them to your Azure VM over SSH



deployment flow · GitHub to Azure VM

Both jobs run on GitHub's machines. The VM only stores the compose file and `.env`, downloads finished images and runs them, so it needs Docker but not Go or Node.

Before you start

Check each item before Step 1.

- [ ] **An Azure account** you can sign in to at portal.azure.com, with credits or a payment method.

- [ ] **The app working locally:** `docker compose up --build` in the `shortener` folder, and http://localhost works.
- [ ] **The code on GitHub,** with `api/go.sum` and `ui/package-lock.json` committed and `.env` not committed.
- [ ] **The `build` job passing** in the repo's **Actions** tab.
- [ ] **PowerShell on your Windows laptop.** Windows 10 and 11 include the `ssh` command.

Step 1: Create the Azure VM

The VM is a Linux computer in Azure's data centre with a public IP address. Your app's three containers will run on it.

1. Go to **portal.azure.com** and sign in.
2. In the search bar at the top, type **Virtual machines** and open it.
3. Click **Create → Azure virtual machine**.
4. On the **Basics** tab, fill in these fields and leave the rest as they are:

Field	Value
Resource group	**Create new** → `shortener-rg`
Virtual machine name	`shortener-vm`
Region	Central India
Image	Ubuntu Server 24.04 LTS
Size	Standard_B1s (enough for this demo)
Authentication type	SSH public key
Username	`azureuser`
SSH public key source	Generate new key pair
Key pair name	`shortener-key`
Public inbound ports	Allow selected ports
Select inbound ports	**SSH (22)** and **HTTP (80)**

5. Click **Review + create**, wait for "Validation passed", then click **Create**.
6. A popup appears. Click **Download private key and create resource**. Your browser saves `shortener-key.pem` to **Downloads**. You can't download it again, so keep it safe.
7. Wait about 2 minutes for "Your deployment is complete", then click **Go to resource**.

8. On the VM's **Overview** page, copy the **Public IP address**, for example `20.198.45.12`. This guide uses that example IP; replace it with yours everywhere.

The resource group is a folder holding the VM and everything Azure created with it. Deleting it later removes them all.

Step 2: Log in to the VM from your laptop

The `.pem` file is the key that opens your VM. Windows' `ssh` refuses to use a key that other users on your laptop could read, and files in Downloads usually are readable. Without the fix below, you get this error:

```
WARNING: UNPROTECTED PRIVATE KEY FILE!
Permissions for 'shortener-key.pem' are too open.
This private key will be ignored.
```

1. Press the Windows key, type `PowerShell` and press Enter.
2. Go to your Downloads folder:

```
cd $HOME\Downloads
```

3. Check that the key is there. It should be listed; if it says "cannot find", look for the file's real name or folder.

```
dir shortener-key.pem
```

4. Remove the permissions the file inherited from the Downloads folder, so nobody can read it:

```
icacls shortener-key.pem /inheritance:r
```

5. Give read access back to yourself only. `$env:USERNAME` fills in your Windows username, and `:R` means read-only:

```
icacls shortener-key.pem /grant:r "$($env:USERNAME):R"
```

Both commands should print `Successfully processed 1 files`.

6. Connect to the VM. `-i` tells ssh which key file to use:

```
ssh -i shortener-key.pem azureuser@20.198.45.12
```

7. The first time, it asks `Are you sure you want to continue connecting (yes/no)?`. Type **yes** and press Enter.

8. When the prompt changes to `azureuser@shortener-vm:~$`, you're inside the VM. Every command until Step 5 runs here.

Steps 4 and 5 are needed once. Next time: open PowerShell, `cd $HOME\Downloads`, then the `ssh` command.

Step 3: Install Docker on the VM

The VM starts as a bare Ubuntu machine. Docker runs your containers; the Compose plugin, which reads `docker-compose.yml`, comes with it.

1. On the VM, download and run Docker's official install script. It takes about a minute:

```
curl -fsSL https://get.docker.com | sh
```

2. Let `azureuser` run Docker without typing `sudo` each time:

```
sudo usermod -aG docker azureuser
```

3. Log out, so the change takes effect:

```
exit
```

4. Log back in from PowerShell with the same command as before:

```
ssh -i shortener-key.pem azureuser@20.198.45.12
```

5. Check that both work:

```
docker ps
docker compose version
```

`docker ps` should print an empty table with headings such as CONTAINER ID, not "permission denied". `docker compose version` should print a version number.

Step 4: Put the app folder on the VM

The VM needs three things the images don't contain: `docker-compose.yml`, the `.env` settings, and permission to download your private images.

4.1 Clone the repo

The workflow runs `cd ~/shortener`, so the folder must have exactly that name and place.

```
git clone https://github.com/<your-username>/shortener.git ~/shortener
```

If your repo is **private**, git asks for a username and password. GitHub doesn't accept your account password here; use a Personal access token instead:

1. On GitHub, open your profile picture → **Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic)**.
2. Name it `shortener-vm`, tick the **repo** scope, and click **Generate token**.
3. Copy the token (it starts with `ghp_`). GitHub shows it only once.
4. Back on the VM, enter your GitHub username, then paste the token as the password. Nothing appears as you paste; press Enter.

So that later `git pull` runs don't ask again, save the token once:

```
cd ~/shortener
git config credential.helper store
git pull
```

Enter the username and token one last time; git remembers them after that.

4.2 Create the .env file

```
cd ~/shortener
nano .env
```

Type these two lines, with your own values:

```
DB_PASSWORD=pick-a-strong-password
GH_OWNER=your-github-username-in-lowercase
```

Save with **Ctrl+O** then **Enter**, and exit with **Ctrl+X**. Check it with `cat .env`.

4.3 Let the VM download your images

The `build` job pushes images to GitHub Container Registry (ghcr.io), where they're private by default. The VM must log in to pull them.

1. Create another classic token as in 4.1, named `shortener-ghcr`, with only the **read:packages** scope.
2. On the VM:

```
docker login ghcr.io -u <your-username>
```

3. Paste the token as the password. You should see `Login Succeeded`.

The alternative is making both images public: on your GitHub profile, open **Packages → shortener-api → Package settings → Change visibility → Public**, then the same for **shortener-ui**.

Step 5: Add the three GitHub secrets

The `deploy` job logs in to your VM exactly as you did in Step 2. These secrets give it the same three things: the address, the username and the key.

Secret name	Value	Where it comes from
`SERVER_HOST`	`20.198.45.12`	VM **Overview** page → Public IP address. Only the IP, no `http://`
`SERVER_USER`	`azureuser`	The username you set in Step 1
`SERVER_SSH_KEY`	the whole `.pem` file	`shortener-key.pem` from Downloads

1. In your repo, open **Settings → Secrets and variables → Actions**.
2. Stay on the **Secrets** tab, not **Variables**, and click **New repository secret**.
3. Add `SERVER_HOST` and `SERVER_USER` with the values above.
4. For `SERVER_SSH_KEY`, copy the key file to your clipboard in PowerShell:

```
Get-Content $HOME\Downloads\shortener-key.pem -Raw | Set-Clipboard
```

5. Paste it as the secret's value. It must include the first and last lines:

```
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEA...many more lines...
-----END RSA PRIVATE KEY-----
```

6. Check that all three names appear under **Repository secrets**. Names are case-sensitive.

You don't add `GITHUB_TOKEN`; GitHub creates it for every run. Never commit the `.pem` file to the repo.

Step 6: Check the workflow file

`.github/workflows/deploy.yml` must contain exactly this. If you changed `deploy` to the laptop version (`runs-on: self-hosted`), replace the whole file.

```
name: Build and Deploy
# The name shown in the Actions tab.

on:
  push:
    branches: [main]
# Runs on every push or merge into main.

jobs:
  build:
    runs-on: ubuntu-latest
    # A fresh GitHub machine, deleted after the job.
    permissions:
      contents: read
      packages: write
      # Allowed to push images to ghcr.io.
    steps:
      - uses: actions/checkout@v4
      # Downloads your code onto the GitHub machine.

      - name: Log in to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push API image
        uses: docker/build-push-action@v6
        with:
          context: ./api
          # Uses api/Dockerfile; Go is compiled here.
          push: true
          tags: |
            ghcr.io/${{ github.repository_owner }}/shortener-api:latest
            ghcr.io/${{ github.repository_owner }}/shortener-api:${{
github.sha }}
          # "latest" is deployed; the commit-hash tag is kept for rollback.

      - name: Build and push UI image
        uses: docker/build-push-action@v6
        with:
          context: ./ui
          # Uses ui/Dockerfile; React is built and packed into Nginx here.
          push: true
          tags: |
```

```
            ghcr.io/${{ github.repository_owner }}/shortener-ui:latest
            ghcr.io/${{ github.repository_owner }}/shortener-ui:${{
github.sha }}

  deploy:
    needs: build
    # Starts only after build succeeds.
    runs-on: ubuntu-latest
    # Another GitHub machine; it only connects to your VM.
    steps:
      - name: Deploy over SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SERVER_SSH_KEY }}
          # The three secrets from Step 5.
          script: |
            cd ~/shortener
            git pull
            docker compose pull
            docker compose up -d --no-build
          # These four commands run ON THE VM:
          # go to the app folder, get the latest compose and config files,
          # download the new images, replace only the changed containers.
```

If your GitHub username has capital letters, pushing to ghcr.io fails. Replace each of the four `${{ github.repository_owner }}` with your username in lowercase.

Step 7: Deploy and check

1. On your laptop, commit the workflow file and get it into `main`, by merging a pull request or with:

```
git checkout main
git pull origin main
git merge my-branch
git push origin main
```

2. Open your repo's **Actions** tab and click the newest **Build and Deploy** run.
3. Wait for `build` to turn green (a few minutes the first time), then `deploy` (under a minute).
4. Open `http://20.198.45.12` (your VM's IP) in a browser. Use `http`, not `https`; HTTPS comes later with a domain.

5. Shorten a URL and click the short link. It should take you to the original page.
6. Try it from your phone on mobile data. Anyone with the IP can now use the app.
7. To see the containers, log in to the VM (Step 2) and run:

```
cd ~/shortener
docker compose ps
```

You should see `db`, `api` and `ui`, all running.

From now on, every push to `main` builds new images and replaces the changed containers on the VM. Saved links survive deployments, because they live in the `pgdata` volume, not in a container.

If something goes wrong

What you see	Why	Fix
`UNPROTECTED PRIVATE KEY FILE`	Key file readable by other users	Run the two `icacls` commands in Step 2
`Connection timed out` when you ssh	Port 22 closed, or wrong IP	VM → **Networking**: allow port 22; recheck the Public IP
`permission denied` on `docker ps`	You didn't log out after `usermod`	`exit`, then ssh in again
`deploy`: "missing server host"	`SERVER_HOST` secret missing or misnamed	Add it under **Secrets** (not Variables), exact name
`deploy`: "ssh: handshake failed" or "unable to authenticate"	Incomplete key or wrong user	Paste the whole `.pem` again; `SERVER_USER` = `azureuser`
`deploy`: "No such file or directory" on `cd ~/shortener`	Repo not cloned on the VM, or in a different folder	Do Step 4.1
`deploy`: `git pull` asks for a password	Token not saved	Do the `credential.helper store` part of Step 4.1
`deploy`: "unauthorized" or "denied" on `docker compose pull`	VM not logged in to ghcr.io	Do Step 4.3
`build`: "denied" when pushing	Username has capital letters	Use your lowercase username in `deploy.yml`

What you see	Why	Fix
Page doesn't load at `http://<IP>`	Port 80 closed, or a container is down	VM → **Networking**: allow port 80; on the VM, `docker compose ps` and `docker compose logs api`

To retry without pushing again: **Actions** tab → the failed run → **Re-run jobs → Re-run failed jobs**.

Roll back to an earlier version

Every image is also tagged with its commit hash, so you can run any earlier version.

1. On GitHub, open the repo's **Commits** and copy the hash of the last good commit, for example `a1b2c3d4...`. Use the full hash.
2. On the VM, open `~/shortener/docker-compose.yml` with `nano` and change `:latest` to `:<that-hash>` on the `api` and `ui` image lines.
3. Run:

```
cd ~/shortener
docker compose pull
docker compose up -d --no-build
```

The database isn't affected. Change the tags back to `:latest` once the fix is pushed; until then, the next `git pull` will report a conflict on that file, which `git checkout docker-compose.yml` clears.

Cost and cleanup

- A running VM is billed by the hour, even when nobody uses the app. Check the **Cost Management** page in the portal, and whether your account has free credits.
- To pause billing for the VM's compute, click **Stop** on its Overview page; **Start** brings it back. The public IP may change after a stop, so update `SERVER_HOST` if it does.
- To remove everything, open **Resource groups → shortener-rg → Delete resource group**, and type its name to confirm. This deletes the VM, its disk and the database with all saved links.

Next steps once it runs: a domain name with HTTPS, then moving Nginx into its own gateway container with load balancing and rate limiting.